PROJECT AND INTERVIEW GUIDE

# IW Computer Vision Services

An AWS-native intelligent document-understanding platform for benefits documents

## What this guide covers

| | |
|---|---|
| **Business problem**<br>Convert inconsistent carrier documents into trustworthy structured benefits and claims data. | **System design**<br>Secure upload, async processing, OCR, classification, strategy routing, validation, review, and delivery. |
| **ML and GenAI**<br>Carrier-aware extraction, table selection research, grounded GenAI, evaluation, and human feedback. | **Career value**<br>FAANG-level resume bullets, metric guardrails, technical interview story, and current delivery status. |

## Simple summary

**The service is like a high-assurance document factory: a user uploads a PDF or image, the platform understands its layout and type, chooses the best extraction method, proves where each result came from, and sends structured data for review and downstream use.**

**How to use this guide**
Read Sections 1-6 to understand the platform. Use Sections 7-10 for Data Scientist / AI-ML Engineer interview preparation. Every performance number is labelled as a documented project result, not a universal production guarantee.

Prepared from repository documentation, code structure, local architecture artifacts, and the supplied project briefs.
Edition: 28 Aug 2026

# 1. Executive summary

**IW Computer Vision Services (CVS)** is an intelligent document-understanding (IDU) platform built for benefits administration. It turns PDFs, scans, and images from carriers into structured, reviewable information such as plan attributes, claims details, and dental line items.

## The problem it solves

Benefits documents are not clean database records. The same business concept can appear in different locations, names, fonts, table layouts, and carrier-specific formats. A purely hard-coded parser is expensive to maintain and breaks when layouts change. A purely generative solution is risky because benefits decisions need traceable evidence.

## The product answer

CVS combines scalable cloud processing with a **hybrid extraction decision**: deterministic rules for stable layouts, OCR and queries for structured evidence, custom ML for table/document selection, and GenAI only inside an evidence-grounded validation boundary. A React console lets operations teams review, correct, author, train, evaluate, and monitor outcomes.

**Hinglish explanation**
Simple language: document upload hota hai, system usko OCR se padhata hai, document type identify karta hai, best model/strategy choose karta hai, result ko source words ke saath validate karta hai, then reviewer and downstream system ko data deta hai.

## Business value

- Reduces repetitive manual document reading while keeping human review for uncertainty and business-critical exceptions.

- Supports carrier-aware change management: new or changed layouts can be handled through versioned templates and strategies instead of rewriting a monolith.

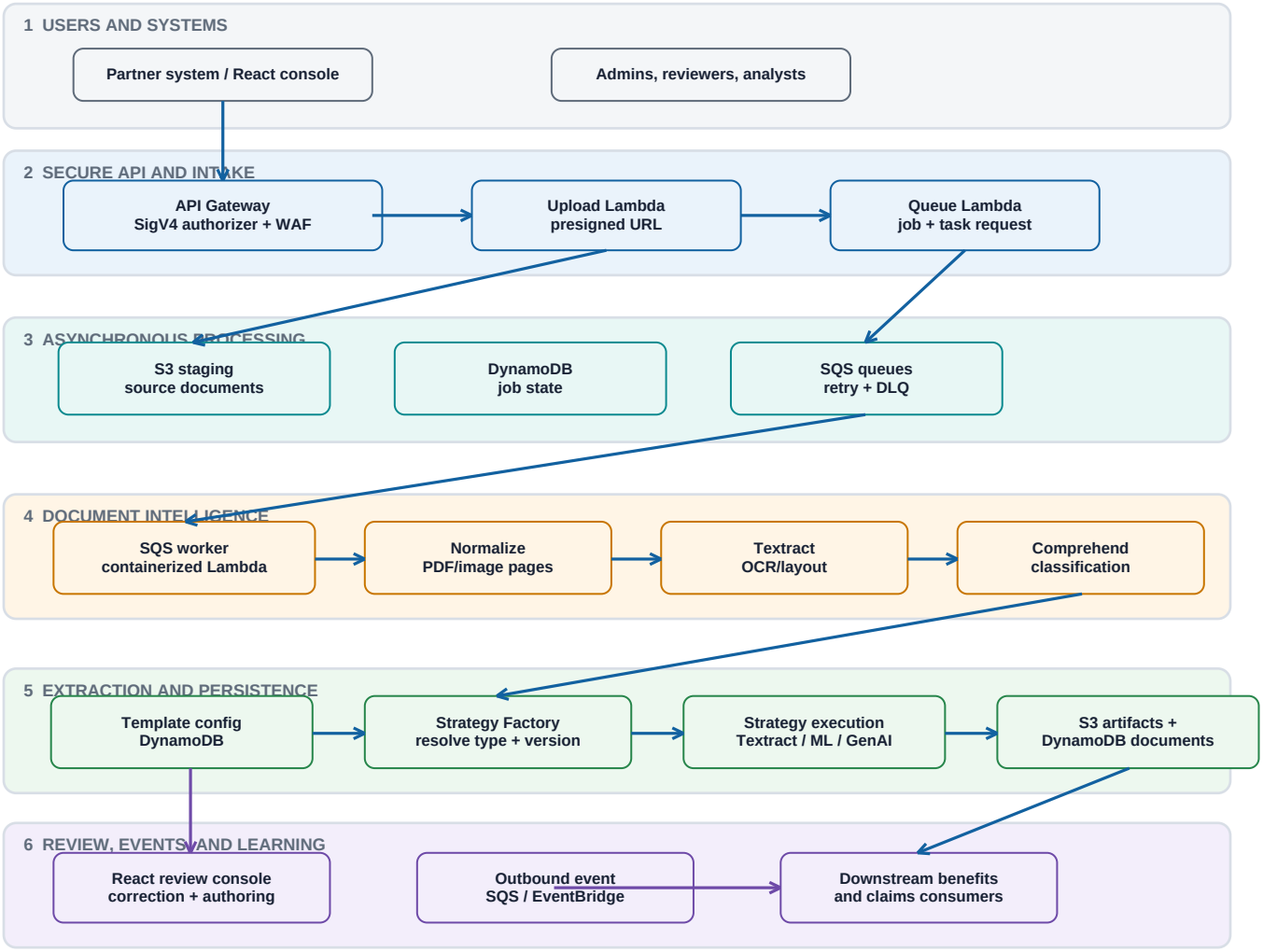
- Creates auditability: extracted values can be tied back to Textract Word IDs, bounding boxes, model version, strategy version, and evaluation evidence.

## Core terms

| Term | Meaning in CVS |
|---|---|
| OCR | Converts page pixels into words, lines, forms, tables, queries, and geometry. AWS Textract is the primary OCR/layout service. |
| Classification | Predicts document type and routing context; AWS Comprehend custom classification is a primary capability. |
| Extraction strategy | Versioned instruction set that says how a document or field group should be extracted. |
| Target | A named output contract, for example a plan field or a line-item group, with ownership and validation rules. |
| Grounding | Each GenAI answer references pre-existing OCR Word IDs; server-side logic verifies them before a value becomes trusted. |

# 2. Top-down architecture

The architecture separates public interaction, secure intake, scalable asynchronous work, document intelligence, and downstream delivery. The split prevents a large PDF or slow OCR call from holding a browser request open.



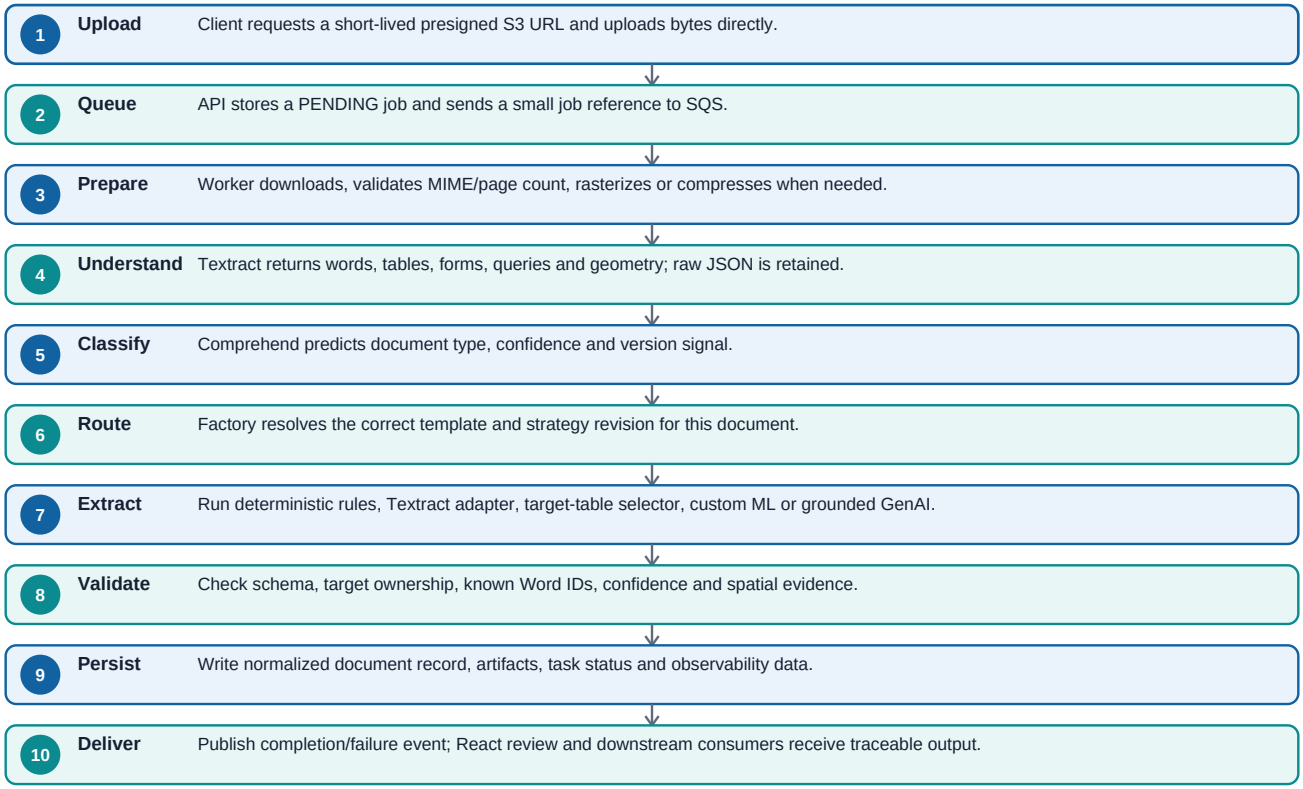

**Design principle**
The queue carries a small job reference, not the document bytes. Documents stay in S3, status stays in DynamoDB, and workers can scale independently. That is a resilient serverless design pattern for document workloads.

## Why each layer exists

| | | |
|---|---|---|
| Users and systems | Partner systems and the React admin console start upload, authoring, testing, and review work. | Browser stays responsive; no direct long-lived AWS credentials. |
| API and intake | API Gateway, WAF, SigV4/session controls, upload and queue Lambdas issue controlled entry points. | Auth and request validation happen before processing. |
| Async processing | S3 holds bytes; DynamoDB owns job state; SQS decouples bursts; DLQ retains unhandled work. | Retries do not duplicate document storage or lose status. |
| Intelligence | OCR, classification, templates, and strategy factory select the correct extraction logic. | One bad document does not block unrelated work. |
| Delivery and learning | Results are persisted, surfaced in UI, emitted to consumers, then reused for supervised improvement. | Review corrections become governed learning evidence. |

# 3. Document runtime flow - from upload to trusted output

This is the most important interview flow. Explain it top to bottom: secure ingestion, asynchronous orchestration, document understanding, strategy selection, validation, and evidence-backed delivery.




## What happens when something fails

**- Retryable/transient error:** the worker can re-queue work, while the job record captures state and diagnostics.

**- Terminal/unhandled error:** the message is isolated in a dead-letter queue (DLQ) for investigation rather than silently disappearing.

**- OCR or classification issue:** raw Textract responses and intermediate artifacts can be retained in S3 so a reviewer or engineer can reproduce the issue.

**- Extraction/GenAI validation failure:** the system rejects unsupported schema, invalid targets, unknown Word IDs, or invalid model ownership rather than trusting a plausible-looking value.

**Interview insight**
Do not describe this as simply 'an OCR model.' It is an end-to-end document-intelligence system that treats OCR as evidence, manages asynchronous work, uses a routing layer, enforces output contracts, and closes the loop through reviewer feedback.

# 4. Component catalog - platform foundations

The repository is organized by operational responsibility. This table maps the principal components to their job in the end-to-end system.

| Component | Primary responsibility | Why it matters |
|---|---|---|
| API Gateway | Public REST boundary for upload, queueing, console operations, templates, strategies and metrics. | Centralizes authorization, WAF protection, CORS/routing and API versioning. |
| Upload Lambda | Creates short-lived presigned S3 upload URLs. | Browser uploads large bytes directly to S3; backend avoids proxying files. |
| Queue Lambda | Creates/updates processing job and places a job reference on SQS. | Fast request response and durable async handoff. |
| S3 staging bucket | Holds incoming PDFs/images; durable bucket also retains normalized pages, Textract JSON and artifacts. | Separates document payload from orchestration state and supports replay/debug. |
| DynamoDB | Stores jobs, classified documents, templates, strategies, models, test results and tasks. | Low-latency state and version metadata for operational workflows. |
| SQS plus DLQ | Buffers inbound processing and isolates unhandled failure messages. | Absorbs bursts, enables retry policy, protects workers and preserves failed work. |
| SQS worker Lambda | Downloads document, calls processing pipeline, updates state and emits completion events. | Elastic per-document processing without a request timeout. |
| Textract | OCR/layout evidence: pages, words, tables, forms, queries, geometry and adapters. | Provides the source material that deterministic, ML, and GenAI methods use. |
| Comprehend custom classifier | Predicts document type and routing confidence. | Routes a heterogeneous document set to the right carrier/document strategy. |

# 4. Component catalog - intelligence, UI, and delivery

| Component | Primary responsibility | Why it matters |
|---|---|---|
| Document template | Defines document type configuration, fields, target contracts and extraction settings. | Moves behavior into governed configuration rather than scattered conditional code. |
| TextractStrategyFactory | Resolves an extraction strategy based on classified type, template, carrier/model revision and runtime context. | Factory pattern isolates algorithm choice and makes future strategies composable. |
| Extraction strategy | Executes queries, forms, table extraction, adapter calls, custom algorithms, composite logic or GenAI. | Each document family can evolve without destabilizing all families. |
| Validation and rehydration | Checks schema, target ownership, Word IDs, evidence, confidence and geometry; reconstructs final fields. | Prevents a weak or ungrounded response from becoming system-of-record data. |
| Outbound queue/event | Notifies downstream benefits/claims consumers of completion or failure. | Decouples consuming systems from processing latency and deployment cadence. |
| React admin console | Review source blocks, correct values, author templates/strategies, choose datasets, run tests and view metrics. | Human-in-the-loop quality and an operational product surface, not a notebook-only prototype. |
| Step Functions | Orchestrates longer model-training/evaluation/testing workflows and state transitions. | Provides visible, retryable workflow coordination beyond a single Lambda invocation. |
| ECS/Fargate | Runs heavier containerized ML/GenAI training or evaluation tasks. | Avoids Lambda runtime limits for CPU/memory/runtime-intensive jobs. |
| CDK | Infrastructure-as-code for API, Lambda, S3, SQS, DynamoDB, ECS, Step Functions, IAM, Secrets and observability. | Makes environment infrastructure reviewable and repeatable. |

## Repository map

| Path | Contents |
|---|---|
| lambda/src | Python REST resources, services, domain objects, repositories, AWS abstractions, extraction strategies and model integrations. |
| cdk/app | Python CDK constructs for code/infrastructure stacks and per-environment configuration. |
| webapp/src | React and TypeScript console for operations, review, authoring, training/test launches and metrics. |
| notebooks | Exploratory R&D; for data analysis, clustering, target-table ML, benchmarking and experiment design. Not all notebooks are production runtime. |
| documentation and root Markdown | Architecture flows, API/design contracts, deployment guidance and experiment documentation. |

# 5. Hybrid intelligence and the factory design

The service deliberately does **not** force every document through one model. The factory pattern separates *what needs to be extracted* from *how to extract it*. This is a practical way to scale a document product across changing carriers and document families.

## Decision ladder

| Method | Use it for | Strength | Control |
|---|---|---|---|
| Textract forms/queries/tables | Known fields and visually structured pages | Fast access to OCR evidence and geometry | Query design, schema and response validation |
| Deterministic strategy | Stable carrier layout or unambiguous header | Predictable and explainable | Versioned strategy and unit/integration coverage |
| Textract adapter | Document family where adapter is available | Managed domain adaptation | Adapter/version tied to strategy |
| Custom table selector | Multiple tables; need exactly one business-relevant table | Ranks tables from semantics, layout and structure | Document-level split, fallback heuristic, calibration |
| Grounded GenAI | Semantic ambiguity, variable language, difficult carrier-specific mapping | Flexible field interpretation with evidence references | Strict JSON/Word-ID/target validator and audit artifacts |
| Composite strategy | Several methods must contribute non-overlapping targets | Reuse and incremental migration | Target collision and dependency-graph checks |

## Factory pattern in this project

**- Input:** classified document type, template configuration, carrier context, strategy/model version, and runtime job state.

**- Resolver: TextractStrategyFactory** selects the strategy implementation rather than a giant if/else block living in the worker.

**- Execution:** a strategy returns fields and/or table groups under defined output targets; a composite strategy can combine safe non-overlapping outputs.

**- Guardrails:** model revision belongs to exactly one strategy, target ownership/collisions are checked, lifecycle versions are controlled, and only eligible revisions should enter runtime selection.

**Why a hiring panel cares**
This is design-for-change. You are showing abstraction boundaries, version governance, testability, extensibility, and an explicit trade-off between deterministic accuracy and model flexibility.

# 6. ML workstreams - target-table selection and clustering

Some benefits documents contain several tables: rates, notes, plan summaries, exceptions, and line-item benefits. The downstream system may need only the line-item table. Selecting the wrong table can make every cell-level extraction look wrong even if OCR is perfect.

## Target-table selection approach

| Stage | Method |
|---|---|
| Candidate creation | Use Textract table output and profile each candidate: headers, aliases, nearby context, row/column count, amount/date/description signals and repeating-row patterns. |
| Baseline | Weighted heuristic scores header match, structural patterns and business keywords. It remains the explicit fallback while models are validated. |
| Training labels | The documented dental dataset contains 200 documents, 653 tables and four categories; one target table per document yields 200 positives and 453 negatives. |
| Leakage-safe evaluation | Split by document ID, not individual table rows. The documented plan uses group-aware train/validation/test splits so tables from one document do not leak across cohorts. |
| Models explored | Logistic Regression, Random Forest, XGBoost, SVM and HistGradientBoosting are documented candidate models. Optimize F1 and compare precision/recall, ROC-AUC, average precision and document-level top-1 accuracy. |
| Runtime plan | Build same features online, score candidates, apply threshold/calibration, choose top table when confident, otherwise use the heuristic/manual-review fallback and log selection evidence. |

## Documented target-table result - use with context

**Offline evaluation result**
The supplied project metrics report F1, precision and recall of 1.0 for dental table selection on the documented evaluation set: 200 documents, 653 tables, four categories. Present this as a cohort-specific offline result, not as an unrestricted production guarantee.

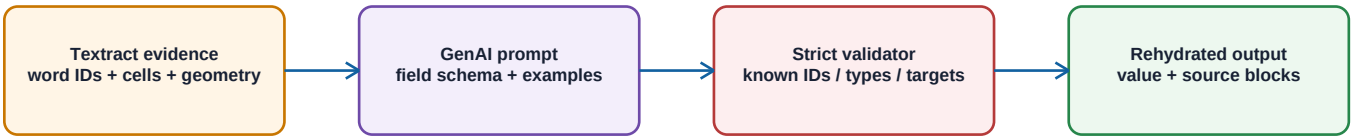
## Unsupervised clustering R&D;

The notebooks also describe an unsupervised discovery path: create document/block features from layout, semantic embeddings and entity frequency; reduce dimensionality (for example UMAP); cluster using KMeans or DBSCAN; assess silhouette and Davies-Bouldin; then inspect clusters for new document families or label-efficiency opportunities.

**Claim boundary**
The notebooks label clustering and some target-table work as exploratory/future productionization. Say 'built/evaluated R&D; pipeline' unless a deployed runtime, reproducible experiment and release evidence confirm production use.

# 7. Grounded GenAI - flexibility without losing evidence

GenAI is most valuable where fields are semantically complex or layouts vary enough that a fixed rule becomes brittle. CVS limits this flexibility with a hard evidence contract: the model must identify source Word IDs already emitted by Textract.




The model may select evidence, but server-side validation decides what becomes a trusted field.

## End-to-end guardrail

- A prompt includes only the relevant document/table/cell evidence, the requested output schema, field definitions, examples, and existing Word IDs.

- The model returns an allowed structured response - for example, field-to-list-of-Word-IDs - rather than arbitrary free text.

- Server code validates response shape, list types, target ownership and every returned Word ID against the original Textract response.

- The system rehydrates trusted output from original OCR words, preserving bounding boxes, confidence and source blocks for UI review and spatial evaluation.

- Unknown IDs, malformed fields or schema violations are rejected and recorded; the UI can show the actual source evidence instead of an unsupported narrative answer.

## Why this matters for regulated or high-impact data

| Risk | Grounded GenAI response |
|---|---|
| Hallucinated value | A model cannot introduce a value without selecting an existing OCR Word ID; server validates it. |
| Wrong field mapping | Target/schema ownership rules and field-level validation constrain placement. |
| No audit trail | Stored artifacts preserve raw OCR, chosen Word IDs, geometry, model/strategy revision and test results. |
| Layout change | Carrier profile/strategy revision can be versioned and evaluated without rewriting unrelated document types. |
| Model regression | Separate test cohorts, evaluation artifacts and promotion lifecycle prevent casual production replacement. |

**How to say it in an interview**
I did not treat an LLM answer as truth. I used GenAI as a constrained evidence selector, then applied deterministic server-side validation and rehydration so every business value remained traceable to the original document.

# 8. Human-in-the-loop product and learning loop

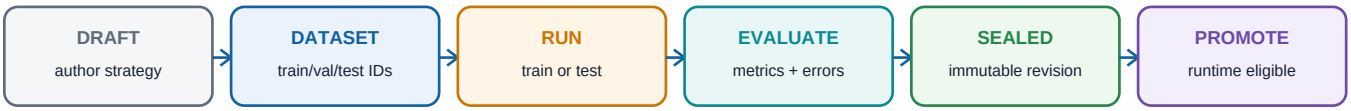
The React console turns the platform into an operational system. It is where users inspect documents, see source evidence, correct extraction, manage templates/strategies, assemble datasets, run evaluation and understand model quality.

## What the console enables

| [illegible] | [illegible] |
|---|---|
| Document review | Review extracted values with source blocks/geometry, correct uncertain results and unblock business processing. |
| Template authoring | Create or update document-type configuration rather than embedding every carrier rule in application code. |
| Strategy/model authoring | Manage extraction strategy, target ownership, adapter or GenAI configuration and version lifecycle. |
| Dataset curation | Choose train/validation/test document IDs and keep cohorts separate for honest evaluation. |
| Evaluation dashboards | Inspect confusion matrix, thresholds, radar/histogram summaries, field failure matrix and qualitative error examples. |
| Operational status | Follow async task status and results without freezing a browser request. |

## Closed learning loop

Review corrections and error cases are not just operational fixes. When governed, they become labeled data for retraining and evaluation. The key is to preserve cohort separation: a document corrected in production should not quietly appear in a held-out test cohort used to claim performance.




Invariant: only sealed, evaluated revisions can be promoted into document runtime selection.

## Lifecycle semantics

- **DRAFT:** configuration can evolve while the strategy/model is being prepared.
- **DATASET/TRAIN/TEST:** explicitly selected cohorts and async tasks create reproducible artifacts.
- **EVALUATE:** compare metrics and inspect failure examples; do not promote based on one aggregate number.
- **SEALED:** immutable revision after required work is complete; this is an audit and reproducibility boundary.
- **PROMOTE:** make an approved, evaluated revision eligible for runtime selection.

# 9. Reliability, governance, and security

A high-quality ML platform is not only its model. CVS contains operational controls that let the team investigate failures, reproduce decisions, protect data and make releases safer.

## Governance model

| Area | Controls | Practical outcome |
|---|---|---|
| Data | Human corrections/labels, distinct train/validation/test cohorts, dataset identifiers. | Honest evaluation and a trail from output back to data. |
| Model | Model IDs, strategy/model versions, kind, owner, test status, metric artifacts and immutable sealed revisions. | A runtime result can identify exactly what logic produced it. |
| Extraction | Output targets, collision detection, dependency graph validation and field/schema contracts. | Composite logic cannot silently overwrite another component's result. |
| GenAI | Prompt/profile/schema/evidence contract, known Word-ID check and rejected-output handling. | Flexible language processing does not bypass deterministic trust controls. |
| Operations | Job/task state, SQS retry, DLQ, logs, artifacts, error status and asynchronous execution records. | Failures are observable and recoverable rather than becoming missing data. |

## Security architecture

- API Gateway is the controlled edge; documented design includes SigV4 authorization and WAF protection.

- Browser upload uses presigned S3 URLs, which reduce backend file-proxy load and can be limited in scope/time.

- The admin console uses same-origin/session-oriented access; it should not hold long-lived AWS or CAS credentials in browser code.

- IAM roles are scoped per service; Secrets Manager is used for protected configuration; CDK makes resource permissions reviewable as code.

- AWS SSO and a named profile are used for environment access during developer tooling, separating local identity from application runtime roles.

**System-design framing**
The platform favors least privilege, short-lived access, durable state, explicit retries, immutable audit artifacts and role-separated orchestration. These are the same principles used in production ML systems at scale.

# 10. Metrics - what to measure and how to state results

Metrics must reflect the actual business question. A document can have high OCR quality and still fail if the wrong table is selected or a value is placed in the wrong target. Track metrics across the full pipeline.

## Metric stack

| Level | Recommended metrics | Why it matters |
|---|---|---|
| Platform | documents/week, queue age, processing latency, error/DLQ rate, cost/document, retry rate | Capacity, reliability and unit economics |
| Classification | macro/micro F1, per-class precision/recall, confidence calibration, confusion matrix | Safe routing across document types |
| Table selection | document top-1 accuracy, precision, recall, F1, ROC-AUC, average precision | Avoids wrong-table cascading errors |
| Extraction | field precision/recall/F1, document pass rate, normalized exact match, target accuracy | Measures actual business data quality |
| Grounding | valid Word-ID rate, evidence coverage, spatial overlap/geometry checks, schema rejection rate | Makes GenAI output auditable |
| Human operations | manual review %, correction rate, time-to-resolution, reviewer agreement | Measures automation value and label quality |

## Documented project figures - include the context

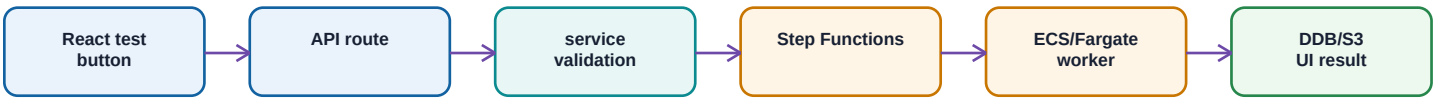
| Reported figure | How to describe it correctly |
|---|---|
| Tens of thousands of documents/week across 20+ classes | Documented platform scale/context; cite environment period and class definition in a formal case study. |
| High 0.8-0.9 classification F1 | Reported project range; attach class distribution, evaluation split, threshold and date. |
| 0.7-0.9 field extraction performance | Reported field/location range; do not collapse heterogeneous fields into one universal accuracy number. |
| Under 10% manual review and under 5 seconds/document | Operational target/result described in supplied brief; define workload, percentile and clock start/end. |
| About $0.03/document for GPT flow | Cost estimate depends on prompt, tokens, model, OCR and document mix; report period and included services. |
| 90%+ grounded GenAI in cited carrier scenarios vs 65-70% rule baseline | Carrier/cohort-specific documented comparison. State fields, sample size and evaluation protocol. |
| Target-table F1/precision/recall 1.0 | Offline dental cohort: 200 documents, 653 tables, four categories. Report alongside held-out split and leakage controls. |

**Resume safety rule**

Never put an accuracy/cost/time number on a resume unless you can explain the denominator, cohort, split, time period and your direct contribution. Strong candidates make metrics more credible by stating their measurement method.

# 11. Current GenAI model-test delivery - what has been implemented

The current feature connects an extraction-strategy test button to a long-running GenAI evaluation workflow. It is intentionally asynchronous because a model test can process a test document cohort and produce artifacts rather than returning inside one HTTP request.




Asynchronous by design: request returns quickly; status and results are persisted for polling/review.

## Implemented integration path

| | |
|---|---|
| React client/workspace | Calls a model-test endpoint from the extraction-strategy workspace; registers async task status and refreshes result state. |
| Lambda API resource | Exposes POST /v1/extraction-strategies/{strategy}/{version}/models/{model}/{version}/test and delegates to service layer. |
| Service validation | Checks async repository, context, GENAI model kind/ownership, non-empty test dataset and no conflicting in-progress evaluation before start. |
| Step Functions | Starts a named GenAI model-test state machine using environment-configured ARN and execution input. |
| ECS/Fargate worker | Runs isolated selected-model evaluation on test documents, persists artifacts/results and completes lifecycle state. |
| DynamoDB/S3/UI | Task/evaluation/result records and artifacts support status, troubleshooting and user-facing test outcome. |

## Deployment and test gate

- CDK synthesis/listing demonstrated the dev environment can resolve the code and expected CVS infrastructure/code stacks.

- The CDK diff should be reviewed in CI: state machine, Fargate task definition, IAM, security groups, API route, Lambda environment variable and webapp artifact must all be present.

- Current status for the new API: **implementation and local diff/test preparation exist; DEV deployment and a live end-to-end execution must be completed before treating it as deployed.**

- The integration test should be opt-in and environment-safe: a real Step Functions ARN, SSO/profile, test documents, permissions and cleanup approach are required. Do not point a local test at production casually.

**Why the 403 happened**
Before the API route/stack is deployed to DEV, API Gateway cannot authorize or route the new resource. A 403 at that stage is a deployment/environment contract issue, not evidence that the browser button or worker logic is wrong.

# 12. Development, testing, and release flow

The deployment architecture is defined in CDK. Local developer commands synthesize/list/diff infrastructure; the repository guidance says actual CDK deployment should be delegated to the approved human/CI release workflow.

## Environment prerequisites

| Requirement | Purpose |
|---|---|
| Python 3.11 virtual environment for cdk | Matches CDK project dependency expectations. A mismatched Python environment can fail before CDK synthesis. |
| AWS_ENV | Selects environment configuration; e.g. bscdatascidev01. The CDK app intentionally fails if unset. |
| AWS SSO profile | Obtains temporary developer credentials, separate from service runtime roles. |
| CDK app command | Invokes the intended virtual-environment Python with app.py so CDK evaluates the same infrastructure code. |
| CI/CD deployment stage | Builds/publishes expected assets and runs the approved deployment role/policy path. |

## Recommended release checklist

- Run targeted unit tests for API client, Lambda service validation, serialization, workflow input and strategy/model repository behavior.

- Run CDK list/synth/diff against the selected dev environment and verify only expected resources/permissions/routes change.

- Use the approved CI branch/pipeline for deployment. Check API route, Lambda env ARN, API Gateway deployment, Step Functions, ECS image/task permissions and webapp asset deployment.

- Execute the guarded integration test only after DEV deploy. Use a known small test cohort and verify Step Functions execution, Fargate logs, DynamoDB task state, S3 artifact, UI result and no unwanted downstream event.

- Monitor CloudWatch/logs, SQS/DLQ, task failures, latency and cost. Capture run ID and artifacts for a reproducible release record.

## Test pyramid

| Test Type | Scope | When to Run |
|---|---|---|
| Unit | Pure strategy/validator/model selection and API request behavior. | Fast; every change. |
| Repository/service | DynamoDB contracts, task lifecycle, ownership/concurrency guards. | Fast or isolated integration environment. |
| CDK synth/diff | Infrastructure and IAM/API contract changes. | Before approved deploy. |
| Live integration | Real API -> Step Functions -> Fargate -> DDB/S3/UI path. | Opt-in DEV only; protected credentials and cleanup. |

# 13. FAANG-level resume bullets - Data Scientist / AI-ML Engineer

Use only bullets that match your personal contribution. Replace bracketed placeholders with verified values and be ready to explain the dataset, validation split, baseline and operational trade-off.

## Strong technical bullets

| Theme | Resume-ready wording |
|---|---|
| Document AI platform | Designed and implemented an AWS-native intelligent document-understanding pipeline using S3, SQS, Lambda, DynamoDB, Textract, Comprehend, Step Functions and ECS/Fargate to convert unstructured benefits documents into traceable structured data. |
| Extensible design | Built a factory-based extraction architecture that resolves versioned carrier/document strategies at runtime, replacing monolithic hard-coded routing with composable deterministic, adapter, custom ML and GenAI components. |
| Grounded GenAI | Implemented evidence-grounded GenAI extraction in which model outputs reference Textract Word IDs; added schema, ownership and source validation before rehydrating reviewer-visible values with geometry and confidence. |
| Async evaluation | Delivered an asynchronous model-test path from React through API/Lambda validation to Step Functions and Fargate, persisting evaluation tasks, artifacts and results for reproducible model governance. |
| Table ML | Developed and evaluated document-level target-table selection features using header semantics, layout/context and structural signals; used grouped splits to prevent document leakage and benchmarked classifiers including XGBoost/HistGradientBoosting against a heuristic fallback. |
| MLOps/quality | Defined a governed strategy/model lifecycle with dataset cohort separation, versioned artifacts, field-level evaluation, promotion gates and operational failure handling (retries, DLQ, audit records). |

## Impact bullets - attach only verified metrics

| Type | Use if you have |
|---|---|
| Target-table evaluation | Achieved [F1/precision/recall] on a held-out, document-grouped dental table-selection cohort of [N documents / N tables], improving reliable line-item extraction by selecting the correct source table. |
| Operational automation | Reduced manual-review rate to [X%] and processing latency to [Y sec/doc] across [volume/time window], while retaining evidence-backed reviewer correction for exceptions. |
| GenAI quality | Improved carrier-specific extraction from [baseline] to [result] on [fields, cohort, date], using Word-ID grounding and schema validation to preserve auditability. |
| Cost | Delivered a GenAI-assisted flow at approximately [$X/document] for [model, page/token/document mix], with [latency] and [quality] guardrails. |

**Do not overclaim**

Do not write 'replaced AWS Textract with a CNN' unless a production deployment, benchmark dataset, reproducible training run and side-by-side metric are available. Safer and stronger: 'researched/evaluated custom CV or CNN alternatives for table detection, with deployment pending validation.'

# 14. Interview narrative - clear, senior, and credible

## 30-second version

**Pitch**
I worked on an AWS-native document-intelligence platform for benefits documents. It ingests PDFs asynchronously, uses Textract for OCR/layout and Comprehend for document routing, then a factory selects a versioned extraction strategy - deterministic, ML, adapter or evidence-grounded GenAI. The important part is trust: results are validated against source Word IDs and shown to reviewers with geometry, so the system scales automation without losing auditability.

## 90-second system-design version

The challenge was carrier variation: templates and table layouts change, so a single hard-coded parser was not maintainable. I separated payload, state and work: S3 stores documents/artifacts, DynamoDB owns job and model metadata, SQS decouples processing, a worker runs OCR/classification and a strategy factory routes to the right algorithm. For difficult semantics, the GenAI path is grounded in Textract Word IDs. The backend validates IDs, schema and target ownership before output is persisted. The console supports review and corrections, while versioned datasets, strategy/model revisions, Step Functions and Fargate give us a repeatable test/evaluation lifecycle. I would measure quality at classification, table-selection, field and evidence levels - not one generic accuracy number.

## Likely technical questions and strong answers

| Question | Answer direction |
|---|---|
| Why not use an LLM for every document? | Cost, latency, nondeterminism and auditability. Use deterministic/OCR methods where reliable; reserve grounded GenAI for semantic ambiguity and validate outputs. |
| How do you prevent data leakage? | Split table-selection/evaluation data by document ID/group, preserve separate train/validation/test cohorts and version the dataset membership. |
| How would you handle a new carrier layout? | Classify/cluster the new samples, author a template/strategy revision, collect labels, test on a held-out cohort, compare against baseline, seal/prompt only after gates pass, then promote. |
| How do you debug an incorrect field? | Trace job/task ID -> model/strategy/template revision -> raw Textract artifact -> selected evidence/Word IDs -> validator output -> UI source block. Then classify root cause: OCR, routing, selection, prompt, schema or target contract. |
| What makes the system scalable? | Direct-to-S3 upload, SQS buffering, stateless workers, persisted state, retries/DLQ and asynchronous Step Functions/Fargate for long-running work. |

## What to emphasize for FAANG interviews

- Start with the business constraint and trade-offs, not a cloud-service list.
- Explain evidence-grounding as a product-quality mechanism, not merely prompt engineering.
- Use leakage-safe evaluation and per-field/operational metrics to demonstrate ML rigor.
- Show how the factory/version model lowers marginal cost of supporting the next carrier or document type.

# 15. Evidence map and next actions

This guide is a technical synthesis. It separates verified repository architecture from documented project results and exploratory notebook work. That distinction protects your credibility in interviews and written materials.

## Primary evidence reviewed

| Source | Used for |
|---|---|
| Repository README.md; lambda/README.md | Platform purpose, module layout, runtime flow, AWS services and inference responsibilities. |
| cdk/README.md; cdk app constructs | CDK environment configuration, code/infrastructure stack separation and deployment policy. |
| PIPELINE_FLOW_DIAGRAM.md | Upload, queue, S3/DynamoDB/SQS worker, Textract/Comprehend/factory/strategy/output flow and failure states. |
| Documentation on automatic table selection and target-table notebooks | Feature engineering, document-level group split, candidates, metrics, heuristic fallback and productionization boundary. |
| Clustering notebook README | Unsupervised discovery research design and its R&D;/future status. |
| Supplied metrics/governance and project-brief PDFs | Reported scale/quality/cost figures, governance lifecycle, grounded GenAI and interview framing. |
| Current API, CDK, React and ECS GenAI model-test code | New test button -> API -> Step Functions -> Fargate -> persisted result delivery status. |

## Recommended next actions before using public numbers

- Create a one-page experiment record per claimed metric: code/revision, data snapshot, group split, cohort dates, label definition, baseline, result and known limitations.

- Complete approved DEV deployment, then run the guarded end-to-end GenAI model-test integration against a small known test cohort.

- Capture CloudWatch/task/DynamoDB/S3 evidence and record a release run ID. Add screenshots only after removing sensitive document content.

- For custom CNN/table-detection claims, locate the exact notebook/model artifacts, record dataset source/licensing, benchmark against the Textract/heuristic baseline and state whether it is research, shadow mode or production.

- Tailor two resume versions: one emphasizing applied ML/evaluation, another emphasizing distributed cloud architecture and MLOps. Both should keep the evidence/metric qualifier.

**Final takeaway**
IW Computer Vision Services is best presented as a trustworthy document-intelligence platform: scalable ingestion plus hybrid ML/GenAI plus a governed human feedback loop. Your strongest differentiator is the combination of system design, ML rigor and evidence-backed AI output.

**End of guide**